

RAG 知识库文档分块与优化：面试详细背诵稿

一、先背这个总纲

如果面试官问：

“你这个知识库文档是怎么处理的？”

可以先这样回答：

我们知识库的文档处理不是简单地上传后按固定字符切，而是把它拆成了解析、分块、图生文、向量化、索引入库和检索召回几个阶段。

整体流程是：用户上传文件后，后端先创建文档记录，状态标记为导入中，然后通过异步任务处理文档。文档处理器会根据文件类型选择不同的解析方式，比如 PDF、DOCX、TXT 属于文本解析型文档，会先提取文本、标题、图片等内容，再做层级分块或递归分块；PPT、Excel、CSV 属于结构化文档，会尽量按照它原有结构，比如 PPT 按页、Excel 按行列或表格区域来处理。

分块完成后，会把每个 chunk 作为检索的基本单位，分批调用 embedding 模型生成向量，再写入向量库，同时可以建立关键词索引。查询时再结合向量检索、关键词检索、混合检索、相似度阈值、top_n 和 rerank 来提升召回效果。

这一段是你的“总入口”。别一上来陷入 PDF 细节，先讲全链路。

二、为什么分块是 RAG 优化？

1. 口语回答

分块属于 RAG 优化，而且是最前置的一环。因为 RAG 后面召回的单位不是整篇文档，而是一个个 chunk。chunk 切得好不好，直接决定能不能召回到正确上下文。

如果 chunk 太小，可能把标题和正文切开，语义不完整；如果 chunk 太大，又会把很多无关内容混在一起，影响相似度计算，也会占用 prompt 上下文。所以分块的目标不是单纯满足长度，而是尽量保证语义完整、结构清晰、来源可追溯。

2. 你要理解的本质

RAG 链路是：

原始文档
↓
解析
↓

分块 chunk
↓
embedding
↓
向量库 / 关键词索引
↓
检索召回
↓
大模型回答

所以如果分块差，后面所有优化都很难救。

比如原文是：

应急领导小组由总经理担任组长，安全总监担任副组长。

如果被切成：

chunk1: 应急领导小组由总经理担任组
chunk2: 长，安全总监担任副组长

用户问：

应急领导小组组长是谁？

这时候 chunk1 和 chunk2 都不是完整语义，embedding 语义也会变弱，最后可能召回不到正确片段。

所以你可以说：

我理解分块优化的核心，是在 chunk_size 限制内尽量保持语义完整性，减少无关噪声，同时保留标题、页码、图片、表格等上下文信息。

三、文档类型怎么分？

1. 面试回答

我会先按文件类型区分处理方式，不同文件不能都按一种方式切。

比如 PDF、DOCX、TXT、Markdown 这类文档，主要是文本解析型文档，会先提取文本内容，再做层级分块或递归分块。

而 PPT、Excel、CSV 这类文档属于结构化文档，它们天然有自己的结构。PPT 更适合按页或者幻灯片切，

Excel 更适合按 sheet、表头、行列区域切，CSV 可以按表头加若干行作为一个 chunk。这样可以尽量保留原始文档结构。

2. 为什么要这么分？

因为文档不是都长一个样。

PDF：版式文档，重点是页面、坐标、页码、图片。
DOCX：有段落、标题样式、表格、图片。
TXT/MD：主要靠换行、标题符号、段落分隔。
PPT：天然按幻灯片组织。
Excel/CSV：天然按表格行列组织。

你可以补一句：

如果所有文档都用固定字符切，简单是简单，但会破坏文档本身的结构，尤其是 PDF、表格和 PPT，召回效果会明显变差。

四、分块策略怎么讲？

1. 固定长度分块

固定长度分块就是按 chunk_size 硬切，比如每 800 或 1000 个字符切一个 chunk。

它的优点是实现简单，适合 demo 或纯文本场景；缺点是容易切断句子、标题、条款和表格，所以真实知识库一般不会只用固定长度分块。

面试官追问：“那你是不是一开始就是这么做的？”

你可以说：

基础版可以先这样做，因为能快速跑通上传、embedding、检索和问答链路。但后续我会把它升级成递归分块和层级分块，避免硬切带来的语义断裂问题。

2. 递归分块

递归分块是你最应该会讲的。

口语版

递归分块不是直接按字符切，而是按分隔符优先级逐级尝试。比如先按双换行切，尽量保持段落完整；如果切出来的段落还是太长，就继续按单换行切；如果还太长，再按空格切；最后实在没办法才按字符切。

它的好处是在满足 `chunk_size` 的前提下，尽量保持自然语言边界，比如段落、句子、词语，不要一上来就把文本切碎。

技术版

可以这样讲：

递归分块会维护一个分隔符优先级列表，比如：

```
["\n\n", "\n", " ", ""]
```

含义是：

1. 优先按双换行切，保持段落完整；
2. 段落过长时按单换行切；
3. 再不行按空格切；
4. 最后用空字符串做字符级兜底。

每次切完后会判断每个片段长度，如果小于 `chunk_size`，就放入待合并队列；如果大于 `chunk_size`，就用下一级分隔符继续递归切分。最后再把小片段智能合并成接近 `chunk_size` 的 `chunk`。

你要记住这个词

不是硬切，是“优先保留语义边界，最后字符兜底”。

3. `chunk_overlap` 怎么讲？

口语版

`chunk_overlap` 是相邻 `chunk` 之间保留一部分重叠内容，主要是为了避免关键信息刚好被切在两个 `chunk` 的边界上。

比如：

```
chunk_size = 800
chunk_overlap = 100
```

代表：

每个 chunk 大约 800 字符，相邻 chunk 之间重复 100 字符。

为什么需要 overlap?

因为真实文本的语义经常跨段落或跨句子。如果完全没有 overlap，前一个 chunk 的结尾和后一个 chunk 的开头可能断开。适当 overlap 可以提高上下文连续性。

overlap 太大会怎样?

overlap 不能太大。太大会导致 chunk 数量变多、向量存储成本上升、检索结果重复、prompt 里塞入重复内容。所以一般会控制在 chunk_size 的 10%-20%左右，具体要看文档类型和评测效果。

面试官追问：“重叠是怎么实现的？”

你可以说：

可以理解为滑动窗口。当前 chunk 拼到接近 chunk_size 后输出，然后从当前 chunk 尾部保留一部分内容，作为下一个 chunk 的开头，再继续拼接后面的片段。这样相邻 chunk 之间就有一段上下文重叠。

4. 层级分块

口语版

层级分块是基于文档结构来切，比如识别 Markdown 标题、Word 标题样式、PDF 目录标题，把文档组织成树状结构，然后按照章节层级进行分块。

比如：

公司制度

第三章 应急管理

应急领导小组

应急领导小组由总经理担任组长……

层级分块后，不只是保留正文，还会保留标题链：

公司制度 > 第三章 应急管理 > 应急领导小组

然后 chunk 内容可以变成：

标题路径：公司制度 > 第三章 应急管理 > 应急领导小组
内容：应急领导小组由总经理担任组长……

为什么层级分块更适合企业文档？

企业制度、规范、手册通常都有章节结构。如果只召回一句话，模型可能不知道这句话属于哪个制度、哪个章节。层级分块可以把父级标题链带到 chunk 里，让召回结果带有上下文。

面试官追问：“如果没有标题怎么办？”

你可以说：

如果文档没有明显标题，就降级到递归分块。也就是优先用结构，结构不存在就用段落、换行、空格等自然边界切，最后再按字符兜底。

五、PDF 分块怎么详细讲？

PDF 是最容易被追问的地方，你要重点背。

1. 为什么 PDF 不能简单按字符切？

面试回答

PDF 比普通文本难处理，因为 PDF 本质上是版式文档，不是天然的流式文本。它的内容是按照页面坐标排版的，比如某个文本块在第几页、x/y 坐标是多少，图片在哪个位置。直接 `get_text` 后按字符切，可能会出现阅读顺序错乱、标题丢失、页码丢失、图片丢失、跨页内容混乱等问题。

所以处理 PDF 时，我会先做解析和结构恢复，再做分块，而不是直接按字符切。

2. PDF 处理整体流程

你可以这样讲：

PDF 处理我会分几步：

- 第一，按页读取 PDF；
- 第二，提取每页的文本块 `text blocks`；
- 第三，根据坐标，特别是 Y 坐标，恢复页面内文本顺序；
- 第四，如果 PDF 有 TOC 目录，就解析目录，拿到标题层级、页码和定位信息；
- 第五，把目录标题和正文内容融合，尽量转成类似 Markdown 的结构；

第六，提取图片，为图片生成 UUID，并在原图位置插入图片占位符；
第七，分块时保留 start_page 和 end_page，方便回答时做来源溯源。

这段很重要，背下来。

3. PDF 的 TOC 目录有什么用？

口语版

TOC 就是 PDF 的目录信息。它可以帮助我们知道某个标题是什么层级、在哪一页、对应正文大概从哪里开始。利用 TOC 可以把 PDF 的章节结构恢复出来。

为什么 TOC 对 RAG 有用？

因为 RAG 召回时，如果 chunk 只有正文，没有标题上下文，模型可能不知道这段内容属于哪个章节。把 TOC 标题融合到正文后，chunk 就能带上章节信息，召回更准确。

举例

原始正文可能只有：

由总经理担任组长，安全总监担任副组长。

如果加上标题链：

安全生产应急预案 > 应急组织机构 > 应急领导小组
由总经理担任组长，安全总监担任副组长。

那用户问“应急领导小组组长是谁”，召回概率就更高。

4. PDF 没有 TOC 怎么办？

面试官很容易这么问。

可以这样答：

不是所有 PDF 都有 TOC。如果没有 TOC，就不能强依赖目录结构。这时候可以降级处理：

- 第一，按页提取文本块；
- 第二，根据字号、加粗、位置、行距等启发式规则尝试识别标题；
- 第三，如果标题也不好识别，就按页和段落做递归分块；
- 第四，至少保留页码信息，保证答案可以溯源。

也就是说，有 TOC 就用 TOC 恢复层级；没有 TOC 就退化为按页解析 + 递归分块。

这句话很加分：

我不会假设所有 PDF 都是规范文档，所以解析策略要有降级路径。

5. PDF 页码怎么保留？

口语版

PDF 问答里页码很重要，因为用户需要知道答案来自第几页。所以我会按页解析时，在每页文本之间插入内部页面标记。分块完成后，再根据这些页面标记计算每个 chunk 的 start_page 和 end_page，最后把内部标记清理掉。

举例

解析后的内部文本可能是：

```
<<PAGE_1>>
第一页内容……

<<PAGE_2>>
第二页内容……

<<PAGE_3>>
第三页内容……
```

分块后，如果某个 chunk 同时包含第 2 页和第 3 页内容，那它的元数据就是：

```
{
  "start_page": 2,
  "end_page": 3
}
```

最终展示给用户时可以说：

来源：安全管理制度.pdf，第 2-3 页。

面试官追问：“跨页 chunk 怎么处理？”

你可以答：

如果一个 chunk 里出现多个页面标记，就取第一个页面标记作为 start_page，最后一个页面标记作为 end_page。如果 chunk 里没有新的页面标记，就沿用当前页码。处理完成后，页面标记只用于内部计算，不会出现在最终 chunk 内容里。

6. PDF 图片怎么处理？

口语版

PDF 里经常有流程图、架构图、截图、扫描件。如果只抽文本，这些图片里的知识就丢了。所以解析 PDF 时可以遍历每一页的图片，提取图片数据，为图片生成 UUID，然后在原图位置插入 Markdown 图片占位符。

图片占位符可以是：

```

```

为什么要插入占位符？

因为占位符能保留图片在原文中的位置。后续如果做图生文，可以把图片描述插入到这个位置，这样 chunk 的上下文就比较自然。

比如原文结构是：

下面是系统部署架构图：
[图片]
各组件说明如下……

图生文后可以变成：

下面是系统部署架构图：
图片描述：该图展示了 rag-backend、worker-document、Redis、MySQL、Elasticsearch 等组件之间的调用关系……
各组件说明如下……

这样图片内容也能被 embedding 和检索。

六、图生文怎么讲？

1. 图生文是什么？

图生文就是用 VLM 视觉语言模型对文档中的图片生成文字描述，然后把描述插入到原图片位置，让图片内容也能参与 embedding 和检索。

2. 为什么它属于 RAG 优化？

因为普通 RAG 主要处理文本，但企业文档里很多信息存在于图片里，比如流程图、架构图、组织结构图、系统截图、扫描件。如果不做图生文，这部分信息就无法被向量检索召回。

图生文后，图片变成文字描述，进入 chunk，后续用户问图片里的流程、结构、节点时，就有机会召回。

3. 处理流程怎么讲？

图生文流程一般是：

- 第一，解析文档时识别图片；
- 第二，为图片生成 UUID，保存图片数据；
- 第三，在 chunk 内容中保留图片占位符；
- 第四，调用 VLM 生成图片描述；
- 第五，把图片描述插入到原图片位置；
- 第六，重新生成段落和向量。

4. 单张图片失败怎么办？

图生文一般不应该因为单张图片失败导致整个文档失败。更合理的做法是单图失败只记录错误，保留原图片占位符，继续处理其他图片，保证文档整体导入可用。

这句话很工程化，可以背。

七、分批 embedding 和进度展示怎么讲？

1. 为什么要分批 embedding？

文档分块后，可能会有几十到几百个 chunk。如果一个 chunk 调一次 embedding，请求次数太多，整体会变慢，也更容易遇到接口限流。所以一般会按 batch 处理，比如每 10 个或 20 个 chunk 一批。

比如 100 个 chunk：

一个一个 embedding: 100 次请求
每批 10 个: 10 次请求

2. 分批怎么做进度?

进度本质上看 `completed_chunks / total_chunks`。

比如一共 100 个 chunk，每批 10 个，每完成一批就 `completed_chunks += 10`。前端看到的进度就是 10%、20%、30% 这样变化。

3. 整体任务进度怎么设计?

可以按阶段加权：

解析文档：0% - 30%
文档分块：30% - 40%
embedding：40% - 85%
索引入库：85% - 100%

如果 embedding 处理了一半：

$$\text{总进度} = 40 + 0.5 * 45 = 62.5\%$$

4. 前端怎么展示?

前端可以通过轮询、SSE 或 WebSocket 获取任务状态。后端每完成一个阶段或一个 embedding batch，就更新任务状态，包括 `progress`、`stage`、`completed_chunks`、`total_chunks` 等字段。

5. flush 和 commit 的坑怎么讲?

这个你前面问过，也可能被追问到。

如果进度更新只是 `session.flush()`，前端接口不一定能看到，因为 `flush` 只是把 SQL 发到当前事务里，还没有 `commit`。前端轮询接口通常是另一个数据库连接和事务，看不到未提交数据。

所以如果要让前端实时看到进度，要么进度更新用独立 session 单独 `commit`，要么把进度写到 Redis 这种任务状态缓存里。

八、向量检索、关键词检索、混合检索怎么讲？

1. 向量检索

向量检索是把用户问题 embedding 成向量，然后和 chunk 向量做相似度计算，找语义最相近的内容。

适合：

用户换一种说法提问
语义相似但关键词不完全一致
比如“请假制度” vs “员工休假管理规定”

缺点：

对专有名词、编号、金额、条款号、人名等精确匹配不一定稳定。

2. 关键词检索 / BM25

关键词检索更偏精确匹配，比如基于 Elasticsearch 或数据库全文索引，根据词频、逆文档频率等方式匹配关键词。

适合：

制度编号
文件名
人名
金额
条款号
专有名词
字段名

比如用户问：

GB/T 22239 是什么？

这种问题关键词检索可能比向量检索更稳。

3. 混合检索

混合检索就是同时做向量检索和关键词检索，然后把两路结果合并、去重、排序。它的好处是既能召回语义相关内容，也能召回关键词精确匹配内容。

你可以这样讲：

向量检索解决“意思像不像”，关键词检索解决“字面上有没有”。企业知识库里有名词和制度条款很多，所以混合检索通常比单纯向量检索更稳。

4. 相似度阈值和 top_n

top_n 是控制返回多少个候选 chunk，比如取前 5 个或前 10 个。

similarity_threshold 是相似度阈值，用来过滤低相关结果。如果分数太低，就不应该塞给大模型，否则模型可能根据无关内容胡答。

回答时可以说：

我会通过命中测试调整 top_n 和 similarity_threshold。top_n 太小可能漏召回，太大又会引入噪声；阈值太高可能没有结果，太低又会带入无关内容。

5. rerank 怎么讲？

初步检索出来的 top_n 结果不一定顺序最好，所以可以用 rerank 模型做二次排序。rerank 会拿用户问题和候选 chunk 做更精细的相关性判断，把最相关的内容排到前面。

你可以这么解释：

embedding 检索适合快速召回，但它是粗排；rerank 是精排。先用向量/关键词召回一批候选，再用 rerank 精排，可以提升最终进入 prompt 的上下文质量。

九、命中测试怎么讲？

1. 什么是命中测试？

命中测试就是输入一个测试问题，系统只看检索阶段返回了哪些 chunk、分数是多少、来源是什么，不一定直接看最终大模型回答。

2. 为什么要做命中测试？

因为 RAG 出错要先判断是召回错了，还是生成错了。

如果正确 chunk 根本没召回，那问题在解析、分块、embedding 或检索策略；

如果正确 chunk 已经召回，但大模型回答错了，那问题可能在 prompt 拼接、上下文排序或模型能力。

这个很重要，背下来。

3. 怎么用命中测试优化？

我会准备一批标准问题，比如制度类问题、表格类问题、图片类问题、精确字段问题。然后查看每个问题是否能召回正确 chunk。

如果召回不到，就看是不是 chunk 切得太碎、标题上下文丢失、关键词索引没建好、相似度阈值太高、top_n 太小，或者 PDF 解析时页码/段落顺序有问题。

十、文件更新和删除怎么讲？

1. 文件删除为什么不能只删 document 表？

因为知识库一个文件上传后，不只是有 document 记录，还会生成 chunk、向量数据、关键词索引、图片资源和对象存储文件。如果只删 document 表，向量库里可能还残留旧 chunk，用户提问时还能召回到已删除文件的内容，这就是脏数据召回。

2. 正确删除流程

删除文件时，我会先把文档状态标记为 deleting 或 deleted，让检索立刻过滤掉它。然后异步清理相关数据，包括 chunk 表、向量库、关键词索引、图片资源和对象存储里的原始文件。清理成功后状态变成 deleted，清理失败就标记 delete_failed，支持重试。

3. 文件更新怎么办？

文件重新上传或者重新解析时，不能让新旧 chunk 同时参与检索。可以先删除旧 chunk 和旧向量，再生成新的；或者引入版本号，只让最新版本参与检索。

可以补一句：

更新和删除的核心是避免脏召回，保证检索结果和用户在文件列表里看到的状态一致。

十一、段落编辑为什么要重新向量化？

1. 面试回答

段落是知识库的最小检索单位，每个段落都有自己的向量。如果用户修改了段落内容，原来的向量已经不代表新内容了，所以需要重新生成 embedding，并更新向量库里的对应记录。

2. 如果只是启用/禁用段落呢？

如果只是禁用段落，不一定要删除数据，可以把 is_active 设置为 false，检索时过滤掉。这样保留数据，后续还能重新启用。

如果是删除段落，就要同步删除数据库段落记录和向量库中的段落向量。

十二、面试高频追问模板

追问 1：你说分块优化，具体优化了什么？

主要优化三个点：

第一，语义完整性。通过递归分块，优先按段落、换行、空格等自然边界切，避免硬切句子。

第二，结构上下文。对于有标题结构的文档，优先做层级分块，把父级标题链带到 chunk 里，让 chunk 不只是孤立正文。

第三，来源可追溯。对于 PDF，分块时保留 start_page 和 end_page，回答时可以展示来源页码。

如果文档有图片，还可以通过图生文把图片内容转成文本，提升多模态文档的召回效果。

追问 2：递归分块和层级分块有什么区别？

递归分块主要基于分隔符和长度控制，比如先按段落、再按换行、再按空格、最后按字符。它更偏通用文本切分。

层级分块主要基于文档结构，比如标题、章节、目录。它会先识别标题层级，把文档组织成树，再把标题链和正文组合成 chunk。

简单说，递归分块解决“怎么不切断自然语义”，层级分块解决“怎么保留章节上下文”。如果文档结构明显，优先层级分块；如果结构不明显，就降级递归分块。

追问 3：PDF 为什么要特殊处理？

因为 PDF 是版式文档，不是天然流式文本。文本、图片、表格都是按页面坐标排布的。直接抽文本再切，可能导致阅读顺序错乱、标题丢失、图片丢失、页码丢失。

所以 PDF 要先按页解析文本块，根据坐标恢复阅读顺序；如果有 TOC 目录，就用目录恢复标题层级；然后再做层级或递归分块。同时保留 start_page 和 end_page，方便答案溯源。

追问 4：PDF 没有目录怎么办？

如果没有 TOC，就做降级处理。可以尝试根据字号、加粗、位置、行距等规则识别标题；如果标题识别也不稳定，就按页和段落做递归分块。即使结构恢复不了，也至少要保留页码，保证召回结果可以追溯到原文页。

追问 5：chunk_size 怎么选？

chunk_size 没有绝对值，要根据文档类型、embedding 模型、召回效果和上下文窗口来调。

太小会导致语义不完整，召回碎片化；太大会引入噪声，影响相似度，也会占用 prompt。一般制度类、说明文档可以从 500-1000 字符或对应 token 范围开始，配合 10%-20% 的 overlap，然后通过命中测试调整。

追问 6：chunk_overlap 有什么副作用？

overlap 可以缓解边界语义断裂，但太大会导致 chunk 数量增加、向量存储成本上升、召回结果重复、prompt 中重复内容变多。所以 overlap 要适中，通常作为防止边界丢信息的补偿，而不是越大越好。

追问 7：图生文会不会不准？

会有这个风险。VLM 生成的图片描述可能存在误识别，所以图生文结果应该作为辅助召回信息，不应该完全替代原始图片。实际工程里可以保留图片链接和图片描述，并在答案溯源时让用户能查看原图。

另外，单张图片处理失败不应该影响整个文档导入，可以记录失败原因，继续处理其他段落。

追问 8：混合检索为什么比单向量检索好？

因为向量检索擅长语义相似，但对编号、人名、金额、专有名词、条款号这类精确匹配不一定稳定。关键词检索正好补这个短板。

混合检索同时走向量和关键词两路召回，再合并去重排序，能兼顾语义召回和精确匹配，所以企业知识库通常更稳。

追问 9：RAG 回答错了怎么排查？

我会先拆成两段排查：召回阶段和生成阶段。

第一步看命中测试，确认用户问题有没有召回正确 chunk。如果正确 chunk 没召回，问题可能在文档解析、分块、embedding、检索策略、top_n 或阈值。

第二步，如果正确 chunk 已经召回，但模型还是答错，那就看 prompt 拼接、上下文排序、rerank、答案约束和模型能力。

所以我不会只看最终回答，而是先看召回是否命中正确证据。

追问 10：文件删了为什么还要删向量？

因为向量库和数据库是两套存储。文件删除后，如果只删数据库 document 记录，不删向量库里的 chunk 向量，用户查询时仍然可能召回旧内容，造成脏数据召回。

所以删除时要同步处理 document、chunk、向量库、关键词索引、图片和对象存储文件。更稳的做法是先标记 deleted，让检索立即过滤，再异步清理外部存储，并支持失败重试。

追问 11：你这个项目里到底实现了哪些？

如果你没有全实现，千万别硬吹。可以这样说：

我自己实现的基础版主要包括：文件上传、异步解析、递归分块、`chunk_overlap`、分批 `embedding`、向量入库、检索问答和任务进度展示。

PDF 这块我实现了按页提取文本和保留页码的基础能力。像 TOC 目录融合、图片图生文、表格结构修复这些，我目前是作为企业级优化方向来设计和理解，知道它们解决什么问题，以及后续怎么演进。

这段非常重要。真实、稳、不心虚。

十三、最终强回答：面试直接背这一段

我理解知识库 RAG 的效果，很大程度取决于索引构建阶段，尤其是文档解析和分块。

我的处理思路是，上传文件后先创建文档任务，由异步 worker 负责解析、分块、`embedding` 和索引入库。分块时不会简单按固定字符硬切，而是根据文档类型选择不同策略。比如 TXT/Markdown 可以走递归分块，优先按段落、换行、空格等自然边界切，最后才按字符兜底；DOCX 可以识别标题、段落、表格和图片，把标题样式转成 Markdown 结构，再按层级分块；PDF 因为是版式文档，会先按页解析文本块，根据坐标恢复阅读顺序，如果有 TOC 目录就恢复标题层级，再做层级分块或递归分块，同时保留 `start_page` 和 `end_page`，方便答案溯源。

分块时我会关注 `chunk_size` 和 `chunk_overlap`。`chunk` 太小会丢上下文，太大会引入噪声；`overlap` 可以缓解边界语义断裂，但也会带来重复和存储成本，所以需要通过命中测试调优。

如果文档里有图片，可以在解析阶段提取图片并插入占位符，再通过 VLM 做图生文，把图片描述插入到原图片位置，让图片内容也能参与 `embedding` 和检索。

分块完成后，会分批调用 `embedding` 模型生成向量，写入向量库，同时可以建立关键词索引。查询时支持向量检索、关键词检索和混合检索，再结合 `top_n`、相似度阈值和 `rerank` 提升召回质量。排查效果时，我会先做命中测试，看正确 `chunk` 有没有被召回；如果召回没问题，再看 `prompt` 和生成阶段。

所以我认为分块是 RAG 优化的基础，但不是全部。完整优化还包括解析质量、图片图生文、混合检索、`rerank`、命中测试、文件更新删除时的向量同步清理等。

十四、记忆口诀

你可以按这个顺序背：

先分类：
文本型、结构化型。

再分块：

固定长度、递归分块、层级分块。

讲 PDF：

版式文档、文本块、坐标排序、TOC、标题融合、页码溯源、图片占位。

讲优化：

chunk_size、overlap、图生文、分批 embedding、混合检索、rerank、命中测试。

讲工程：

异步任务、进度展示、删除向量、失败重试。

最后兜底：

基础版我实现了，企业级能力我理解演进方向。